Constraint Satisfaction Problems (CSP)

Wildan Nadiyal Ahsan, M.Kom.
Prodi Sains Data – UIN Raden Mas Said Surakarta

1. Apa itu CSP?

Constraint Satisfaction Problem (CSP) adalah kelas masalah dalam kecerdasan buatan di mana tujuan bukan sekadar menemukan **solusi terbaik**, tetapi menemukan **penugasan variabel yang memenuhi semua constraint** yang ditetapkan.

Secara formal, CSP didefinisikan sebagai triple {X, D, C} di mana:

- **X** = himpunan variabel $\{X_1, X_2, ..., X_n\}$
- **D** = himpunan domain $\{D_1, D_2, ..., D_n\}$ — nilai yang boleh diambil setiap variabel
- **C** = himpunan constraint yang membatasi kombinasi nilai

Solusi CSP adalah penugasan lengkap (setiap variabel memiliki nilai) yang konsisten (semua constraint terpenuhi).

Mengapa CSP Penting?

Aspek	Pencarian Biasa	CSP
Representasi masalah	State atomik (black box)	Terstruktur — variabel dan nilai
Tujuan	Capai state goal	Penuhi semua constraint
Deteksi kegagalan	Hanya di goal test	Lebih awal saat assign nilai
Pruning pencarian	Tidak ada struktur	Propagasi constraint sangat efektif

Contoh CSP Populer

- **Sudoku** — variabel = sel, domain = {1..9}, constraint = baris/kolom/kotak unik
- **N-Queens** — variabel = baris, domain = kolom, constraint = tidak saling serang
- **Pewarnaan Graf** — variabel = node, domain = warna, constraint = tetangga beda warna
- **Penjadwalan** — variabel = kelas, domain = slot waktu, constraint = tidak bentrok
- **Kriptaritmetik** — variabel = huruf, domain = {0..9}, constraint = persamaan aritmetika

2. Komponen CSP

2.1 Variabel

Variabel adalah entitas yang nilainya belum diketahui dan perlu ditentukan. Biasanya dinotasikan sebagai $X_1, X_2, ..., X_n$.

- Bisa merepresentasikan wilayah, sel, posisi, waktu, dsb.

- Jumlah variabel menentukan ukuran ruang pencarian.
- Setiap variabel memiliki domain masing-masing.

2.2 Domain

Domain D_i adalah himpunan nilai yang boleh diambil variabel X_i.

- **Domain diskrit terbatas**: {Merah, Hijau, Biru}, {1, 2, 3, 4, 5}
- **Domain diskrit tak terbatas**: himpunan bilangan bulat
- **Domain kontinu**: interval bilangan real (CSP numerik)

2.3 Constraint

Constraint membatasi kombinasi nilai yang diizinkan, diekspresikan sebagai relasi antara variabel.

Tipe	Deskripsi	Contoh
Unary	Melibatkan 1 variabel	WA ≠ Hijau
Binary	Melibatkan 2 variabel	WA ≠ NT
Ternary/Global	Melibatkan ≥3 variabel	AllDifferent(NSW, Q, V, SA)
Soft	Preferensi (bisa dilanggar dengan penalti)	Preferensi waktu pengajar

2.4 Solusi

Solusi CSP adalah penugasan $\{X_1=v_1, X_2=v_2, ..., X_n=v_n\}$ di mana setiap $v_i \in D_i$ dan semua constraint $c \in C$ terpenuhi. CSP bisa memiliki nol, satu, atau banyak solusi.

3. Representasi Graf Constraint

CSP dengan constraint binary dapat divisualisasikan sebagai graf constraint:

- **Node** = Variabel
- **Edge** = Constraint binary antara dua variabel

Graf ini berguna untuk memahami struktur masalah dan merancang strategi pencarian yang efisien.

Adjacency Peta Australia

Wilayah	Tetangga (constraint ≠)
WA	NT, SA
NT	WA, SA, Q
SA	WA, NT, Q, NSW, V
Q	NT, SA, NSW
NSW	Q, SA, V
V	SA, NSW

Wilayah	Tetangga (constraint ≠)
T	(tidak ada — terisolasi)

Struktur Graf dan Kompleksitas

- **Graf pohon (tree-structured)**: $O(n \cdot d^2)$ — jauh lebih efisien
- **Graf hampir pohon**: bisa dimanfaatkan dengan cutset conditioning
- **Graf umum**: membutuhkan backtracking, worst case eksponensial

4. Contoh: Pewarnaan Peta Australia

Tujuan: warnai peta Australia sehingga tidak ada dua wilayah yang berbatasan mendapat warna yang sama, menggunakan minimal 3 warna.

Formulasi CSP

- **Variabel**: WA, NT, SA, Q, NSW, V, T
- **Domain**: D_i = {Merah, Hijau, Biru} untuk setiap i
- **Constraint**: WA≠NT, WA≠SA, NT≠SA, NT≠Q, SA≠Q, SA≠NSW, SA≠V, Q≠NSW, NSW≠V

Satu Solusi Valid

Wilayah	Warna	Tetangga (harus beda)
WA	Merah	NT(Hijau), SA(Biru)
NT	Hijau	WA(Merah), SA(Biru), Q(Merah)
SA	Biru	WA, NT, Q, NSW, V — semua beda
Q	Merah	NT(Hijau), SA(Biru), NSW(Hijau)
NSW	Hijau	Q(Merah), SA(Biru), V(Merah)
V	Merah	SA(Biru), NSW(Hijau)
T	Merah	(tidak ada tetangga)

Kunci: SA adalah variabel paling kritis — memiliki 5 tetangga. Jika SA tidak diberi warna yang tepat lebih awal, hampir pasti terjadi backtrack. Inilah yang diantisipasi heuristik MRV dan Degree Heuristic.

5. Algoritma Backtracking Search

Backtracking adalah algoritma pencarian standar untuk CSP. Ide dasarnya: assign nilai ke satu variabel pada satu waktu, dan segera backtrack jika terjadi konflik.

Pseudocode

```
function BACKTRACKING-SEARCH(csp):
    return BACKTRACK({}, csp)
```

```
function BACKTRACK(assignment, csp):
    if assignment lengkap:
        return assignment          # Solusi ditemukan!

    var = SELECT-UNASSIGNED-VARIABLE(csp, assignment)

    for value in ORDER-DOMAIN-VALUES(var, assignment, csp):
        if value konsisten dengan assignment:
            assignment[var] = value
            result = BACKTRACK(assignment, csp)
            if result != failure:
                return result
        hapus assignment[var]      # Backtrack

    return failure
```

Implementasi Python

```
def is_consistent(var, value, assignment, constraints):
    for neighbor in constraints.get(var, []):
        if neighbor in assignment and assignment[neighbor] == value:
            return False
    return True

def backtrack(assignment, variables, domains, constraints):
    if len(assignment) == len(variables):
        return assignment  # Solusi ditemukan

    unassigned = [v for v in variables if v not in assignment]
    var = unassigned[0]

    for value in domains[var]:
        if is_consistent(var, value, assignment, constraints):
            assignment[var] = value
            result = backtrack(assignment, variables, domains, constraints)
            if result is not None:
                return result
            del assignment[var]  # Backtrack

    return None

# Setup masalah pewarnaan Australia
variables  = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
domains    = {v: ['R', 'G', 'B'] for v in variables}
constraints = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q' : ['NT', 'SA', 'NSW'],
    'NSW':['Q', 'SA', 'V'],
    'V' : ['SA', 'NSW'],
    'T' : []
}
solution = backtrack({}, variables, domains, constraints)
print(solution)
# {'WA':'R', 'NT':'G', 'SA':'B', 'Q':'R', 'NSW':'G', 'V':'R', 'T':'R'}
```

Kompleksitas

Worst case: $O(d^n)$ — di mana d = ukuran domain, n = jumlah variabel.

Backtracking murni adalah depth-first search di ruang assignment. Tanpa heuristik, kinerjanya bisa sangat buruk. Kombinasi dengan MRV, LCV, dan forward checking dapat mengurangi node yang dieksplorasi secara drastis.

6. Heuristik: MRV dan LCV

6.1 Minimum Remaining Values (MRV)

MRV (juga disebut fail-first heuristic atau most constrained variable) memilih variabel dengan **domain terkecil yang tersisa** untuk di-assign berikutnya.

Intuisi: dengan memilih variabel yang paling "terkekang" terlebih dahulu, dead-end terdeteksi lebih awal dan cabang pencarian yang tidak mungkin dibuang lebih cepat.

```
def select_mrv_variable(assignment, domains, constraints):
    """Pilih variabel dengan domain tersisa terkecil (MRV)."""
    unassigned = [v for v in domains if v not in assignment]
    return min(unassigned, key=lambda v: len(domains[v]))
```

6.2 Least Constraining Value (LCV)

LCV memilih nilai yang paling sedikit mengeliminasi nilai dari domain tetangga. Kita ingin **memaksimalkan opsi tetangga**.

```
def lcv_order(var, assignment, domains, constraints):
    """Urutkan nilai berdasarkan LCV (ascending constraining effect)."""
    def count_conflicts(value):
        count = 0
        for neighbor in constraints.get(var, []):
            if neighbor not in assignment:
                count += sum(1 for nv in domains[neighbor] if nv == value)
        return count
    return sorted(domains[var], key=count_conflicts)
```

Perbandingan MRV vs LCV

Aspek	MRV	LCV
Target	Pemilihan variabel	Pemilihan nilai
Strategi	Fail-first	Succeed-first
Tujuan	Deteksi dead-end awal	Maksimalkan ruang pencarian
Overhead	Rendah	Sedang (hitung konflik)
Dampak	Sangat signifikan	Bergantung masalah

7. Propagasi Constraint: Algoritma AC-3

AC-3 (Arc Consistency Algorithm 3) adalah teknik propagasi constraint yang memperketat domain variabel sebelum atau selama backtracking — sehingga mengurangi ruang pencarian secara signifikan.

Konsep: Arc Consistency

Arc (X, Y) disebut arc-consistent jika untuk setiap nilai x ∈ Domain(X), terdapat setidaknya satu nilai y ∈ Domain(Y) yang memenuhi constraint X ↔ Y.

Pseudocode AC-3

```
function AC-3(csp):
    queue = semua arc (Xi, Xj) dalam csp

    while queue tidak kosong:
        (Xi, Xj) = dequeue(queue)
        if REVISE(csp, Xi, Xj):
            if Domain(Xi) kosong:
                return false          # Tidak ada solusi!
            for Xk in tetangga(Xi) - {Xj}:
                tambah (Xk, Xi) ke queue
    return true

function REVISE(csp, Xi, Xj):
    revised = false
    for x in Domain(Xi):
        if tidak ada y in Domain(Xj) yang memenuhi constraint(Xi, Xj):
            hapus x dari Domain(Xi)
            revised = true
    return revised
```

Implementasi Python

```
from collections import deque
import copy

def ac3(domains, constraints):
    domains = copy.deepcopy(domains)
    queue = deque()
    for var in constraints:
        for neighbor in constraints[var]:
            queue.append((var, neighbor))

    while queue:
        xi, xj = queue.popleft()
        if revise(domains, xi, xj):
            if len(domains[xi]) == 0:
                return False, None  # Domain kosong
            for xk in constraints.get(xi, []):
                if xk != xj:
                    queue.append((xk, xi))
    return True, domains

def revise(domains, xi, xj):
    revised = False
    for x in list(domains[xi]):
        if all(x == y for y in domains[xj]):
            domains[xi].remove(x)
            revised = True
    return revised
```

Kompleksitas AC-3

Waktu: $O(ed^3)$ — Ruang: $O(e)$ — di mana e = jumlah arc, d = ukuran domain terbesar.

AC-3 menjamin arc consistency, bukan path consistency atau global consistency. Setelah AC-3, mungkin masih perlu backtracking. AC-3 paling efektif sebagai preprocessing atau dalam loop backtracking (MAC: Maintaining Arc Consistency).

8. Perbandingan Metode CSP

Metode	Deskripsi	Kelebihan	Kekurangan
Backtracking Dasar	DFS dengan pengecekan konsistensi	Mudah diimplementasi	Sangat lambat tanpa heuristik
BT + MRV	Pilih variabel domain terkecil	Deteksi dead-end lebih awal	Perlu hitung domain size
BT + LCV	Pilih nilai paling sedikit membatasi	Efektif saat banyak solusi	Overhead kalkulasi konflik
Forward Checking	Propagasi lokal saat assign nilai	Deteksi kegagalan lebih dini	Tidak propagasi jauh
AC-3 (preprocessing)	Perketat domain sebelum mulai	Efektif untuk constraint ketat	Tidak cukup untuk semua kasus
MAC	Jalankan AC-3 setelah setiap assignment	Paling efektif untuk banyak masalah	Overhead propagasi tiap langkah
Min-conflicts	Perbaiki konflik secara lokal	Sangat cepat untuk masalah besar	Tidak guaranteed complete

Kinerja Komparatif (N-Queens 8×8)

Metode	Node Dieksplorasi	Perbandingan
BT Dasar	~2.057	Baseline
BT + Forward Check	~877	2.3× lebih cepat
BT + FC + MRV	~113	18× lebih cepat
BT + MAC	~64	32× lebih cepat

9. Latihan Soal

Soal 1 — Formulasi CSP

Formulasikan masalah 4-Queens sebagai CSP. Tentukan variabel, domain, dan constraint-nya. Kemudian tunjukkan bahwa solusi {Q1=2, Q2=4, Q3=1, Q4=3} adalah solusi valid.

Petunjuk: *Gunakan representasi satu variabel per baris, domain = kolom. Constraint: tidak ada dua ratu yang berada di kolom sama atau diagonal sama.*

Soal 2 — Backtracking Trace

Lakukan trace backtracking pada CSP berikut:

- Variabel: A, B, C

- Domain: {1, 2, 3}
- Constraint: $A \neq B$, $B \neq C$, $A + C \leq 4$

Temukan semua solusi (assignment lengkap yang memenuhi semua constraint).

Soal 3 — Sudoku Mini

Formulasikan Sudoku 4×4 sebagai CSP. Tunjukkan bagaimana AC-3 dapat memperketat domain untuk sebuah puzzle yang sudah diberikan.

Soal 4 — Implementasi

Implementasikan backtracking dengan MRV untuk menyelesaikan Sudoku 9×9. Ukur berapa node yang dieksplorasi dengan dan tanpa MRV. Analisis perbedaan kinerja yang Anda temukan.

Soal 5 — Analisis Kompleksitas

Bandingkan kompleksitas waktu tiga pendekatan untuk CSP dengan n variabel dan domain size d:

- Generate-and-test (brute force)
- Backtracking dasar
- Backtracking + AC-3

Dalam kasus apa masing-masing pendekatan lebih unggul?

10. Contoh Nyata: Penjadwalan Kuliah

Penjadwalan kuliah adalah salah satu aplikasi CSP paling relevan dan kompleks dalam praktik. Setiap kampus harus menjadwalkan mata kuliah, dosen, dan ruang dalam slot waktu terbatas.

Formulasi CSP

- **Variabel**: setiap mata kuliah yang perlu dijadwalkan
- **Domain**: (slot_waktu × ruang) — pasangan slot dan ruang yang tersedia
- **Constraint**: tidak ada konflik dosen, ruang, atau mahasiswa di waktu bersamaan

Jenis Constraint

Tipe	Constraint	Contoh
Hard constraint	Dosen tidak boleh dua kelas bersamaan	Dr. Ali hanya di satu ruang per slot
Hard constraint	Ruang tidak boleh dobel-booking	Aula A hanya satu kelas per waktu
Hard constraint	Kapasitas ruang cukup	30 mhs butuh ruang ≥ 30 kursi
Soft constraint	Preferensi dosen	Dosen B tidak mau Jumat sore
Soft constraint	Distribusi merata	Jangan 3 ujian di hari yang sama

Implementasi Sederhana

```
from itertools import product

courses     = ['AI', 'DB', 'ML', 'Web']
instructors = {'AI':'Budi', 'DB':'Citra', 'ML':'Budi', 'Web':'Dani'}
slots  = ['Sen-08', 'Sen-10', 'Sel-08', 'Sel-10', 'Rab-08', 'Rab-10']
rooms  = ['R1', 'R2']
domain = [(s, r) for s, r in product(slots, rooms)]
domains = {c: domain.copy() for c in courses}

def schedule_consistent(c1, v1, c2, v2):
    slot1, room1 = v1
    slot2, room2 = v2
    if slot1 == slot2:
        if room1 == room2: return False
        if instructors[c1] == instructors[c2]: return False
    return True

def solve_schedule(assignment={}, courses_list=courses):
    if len(assignment) == len(courses_list):
        return assignment
    unassigned = [c for c in courses_list if c not in assignment]
    course = unassigned[0]
    for val in domains[course]:
        ok = all(schedule_consistent(course, val, c2, assignment[c2])
                 for c2 in assignment)
        if ok:
            assignment[course] = val
            result = solve_schedule(assignment, courses_list)
            if result: return result
            del assignment[course]
    return None

schedule = solve_schedule()
for course, (slot, room) in schedule.items():
    print(f"{course}: {slot} di {room} (dosen: {instructors[course]})")
```

11. Referensi

1. Russell, S. J., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Prentice Hall. [Bab 5: CSP]

2. Dechter, R. (2003). Constraint Processing. Morgan Kaufmann Publishers.

3. Mackworth, A. K. (1977). Consistency in networks of relations. Artificial Intelligence, 8(1), 99–118.

4. Kumar, V. (1992). Algorithms for constraint satisfaction problems: A survey. AI Magazine, 13(1), 32–44.

5. Tsang, E. (1993). Foundations of Constraint Satisfaction. Academic Press.

6. Apt, K. R. (2003). Principles of Constraint Programming. Cambridge University Press.